Система автоматизации проектирования

архитектуры микросервисных систем

AutoSystemDesign

Дичковский Владимир

Цель

Разработать инструмент для автоматизированного построения и проверки архитектурных моделей микросервисной системы на основе функциональных и нефункциональных требований и заданного профиля нагрузки.

Входные данные и результаты

Пользователь передаёт на вход:

- BRD (документ бизнес-требований)
- Функциональные требования
- Нефункциональные требования
- Профиль нагрузки

Пользователь получает на выход:

- C4 Component Diagram
- Sequence-диаграмма на каждое функциональное требование
- OpenAPI 3.1 и Swagger UI

Вход можно задать в YAML или передать документ BRD: требования из документа извлекаются автоматически.

Выходные артефакты

Диаграмма компонентов

- Скомпилированное изображение в выбранном формате (PNG/JPG)
- Исходный код PlantUML
- JSON доменной модели архитектуры компонентов

Диаграмма последовательностей

- Скомпилированное изображение в выбранном формате (PNG/JPG)
- Исходный код PlantUML
- JSON доменной модели диаграммы последовательно-стей

Спецификация API

- Спецификация OpenAPI в формате JSON
- Swagger UI в формате HTML

Изображения предназначены для просмотра, PlantUML — для редактирования диаграмм, JSON — для повторного использования структурированных моделей.

Варианты сервера рендеринга PlantUML

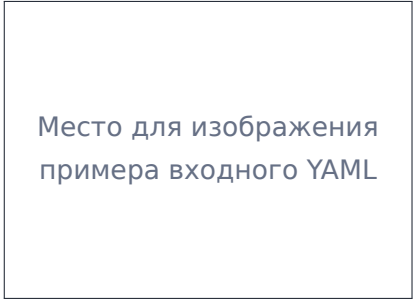
Для компиляции исходного кода PlantUML в изображение доступны три варианта:

1. **Локальный сервер PlantUML, развёртываемый через Docker Compose**
   Развёртывается вместе с проектом; конфигурация включена в репозиторий.
2. **Официальный сервер PlantUML**
   Используется размещённый разработчиками PlantUML сервер.
3. **Kroki**
   Универсальный сервис рендеринга диаграмм, поддерживающий PlantUML и другие форматы.

Пример входного YAML

Структура сценария

```yaml
project_name: "social_network_chat"
description: "Messaging module"
workload:
  baseline_requests_per_second: 500.0
  peak_requests_per_second: 2500.0
  read_ratio: 0.6
requirements:
  - type_identifier: latency
    threshold_value: 150.0
    percentile: 99.0
functional_requirements:
  - identifier: "req_msg_01"
    name: "Direct Message Delivery"
    text: >
      Sender sends a message to a recipient.
```



YAML — один из вариантов входа; требования также извлекаются из документа BRD.

Генерация и проверка

Требования и профиль нагрузки

↓

LLM через LiteLLM

Structured output по классам доменной модели

↓

Типизированные модели и валидаторы

↓

Генерация PlantUML и рендеринг

- LiteLLM позволяет обращаться к разным моделям через OpenAI-совместимый API.
- LLM формирует structured output по классам доменной модели; код PlantUML она не пишет.
- Доменная модель описывает архитектуру, функциональные требования и спецификацию API.
- Рендерер преобразует эту модель в диаграммы; его можно заменить

Повторная генерация при ошибках

1. LLM формирует структурированный ответ по классам доменной модели.
2. Ответ разбирается как JSON и преобразуется в типизированные модели.
3. При ошибке JSON-разбора в контекст добавляется описание ошибки.
4. Если модель не проходит доменные валидаторы, в контекст добавляется форматированное описание нарушенных правил.
5. LLM получает следующую попытку с учётом ошибки и исправляет ответ.

Число попыток ограничено настройкой. Если лимит исчерпан, генерация завершается ошибкой, а не запускается бесконечно.

Модель архитектуры C4

Типы узлов

- Клиент
- Балансировщик нагрузки
- Вычислительный узел
- База данных
- Кэш
- Брокер сообщений

Параметры узла

Фиксированные

- Характеристики и стоимость узлов Amazon EC2 по открытым источникам
- Каталог можно расширять конфигурациями других провайдеров и собственными узлами

Определяемые LLM

- Число реплик
- Параллелизм
- Время обработки
- Размер очереди

Профиль нагрузки (RPS) задаётся во входных данных.

Онтология связей между узлами

Примеры разрешённых связей

- Клиент → вычислительный узел
- Вычислительный узел → база данных

Примеры запрещённых связей

- Клиент → база данных
- База данных → вычислительный узел

Онтология задаёт допустимые пары типов узлов; проверка выполняется до построения диаграммы.

Валидаторы структуры графа

- **Уникальность узлов:** каждый узел имеет уникальный идентификатор.
- **Допустимость связей:** каждая связь соответствует разрешённой паре типов узлов.
- **Отсутствие циклов:** граф не содержит запрещённых циклических зависимостей.
- **Связность:** нет узлов, не связанных ни с одним другим узлом.

Эти проверки оценивают структуру графа и не заменяют проверку смысла взаимодействий.

Валидация семантики и расчётов

Примеры семантических правил

- Брокер сообщений должен быть связан как минимум с узлом, отправляющим сообщения, и узлом, получающим сообщения.
- При использовании кэша должен быть определён путь обращения к основному хранилищу при промахе кэша.

Валидация на основе расчётов

- Рассчитывается загрузка вычислительных узлов с учётом потока запросов, числа реплик и параллелизма.
- Оцениваются теоретическая задержка обработки и соблюдение заданного бюджета задержки.

Протоколы взаимодействия между узлами

Для каждой связи выбирается протокол из разрешённого набора.

- HTTP (REST)
- gRPC
- TCP
- Message Queue

Планируется добавить: Webhooks, Long Polling и Server-Sent Events (SSE).

Для каждого протокола задано, является ли взаимодействие синхронным или асинхронным.

Планируемые валидаторы диаграммы компонентов

Планируемые направления

- Симуляция нагрузки с помощью SimPy.
- Модели на основе теории массового обслуживания.
- Анализ очередей, задержек и насыщения под нагрузкой.

Цель: дополнить статические проверки оценкой поведения модели при заданном профиле нагрузки.

Планируемые возможности, не текущая реализация.

Реализованные валидаторы диаграммы последовательностей

Текущие правила

- На каждое функциональное требование — отдельная диаграмма.
- Ссылки на существующие роли и узлы C4.
- Соответствие сценария исходному требованию.
- Использование всех объявленных линий жизни.
- Разные бизнес-роли не используют один клиентский компонент.
- События и уведомления должны быть асинхронными.

Дальнейшее развитие

Расширение проверок семантики сообщений и ветвлений сценария.

REST API: OpenAPI и Swagger

OpenAPI 3.1

- Операции и маршруты
- Параметры и схемы
- Связь с требованиями и узлами C4

Проверки

- Уникальность операций
- Коллизии маршрутов
- Согласованность ссылок



Интерфейс и варианты развития

Текущая реализация

- Консольный интерфейс для запуска сценариев генерации.
- YAML-конфигурация со всеми настройками приложения, включая параметры LLM и число попыток.

Возможные направления развития

- Добавить FastAPI и запускать генерацию как серверный сервис.
- Разработать отдельный фронтенд для настройки сценариев и просмотра результатов.

Серверный API и отдельный фронтенд — возможные дальнейшие этапы, а не часть текущего интерфейса.